

Sistema que convierte tareas en lenguaje natural en operaciones Git aisladas mediante un pipeline determinístico de transformación semántica.

Principio: *LLM for intention; deterministic system for execution.*

```

Usuario → /agent/interpret → Draft → /agent/confirm → Plan →
/agent/apply → Worktree + verify
              (LLM + catálogo)          (humano edita)          (PlanCompiler)
(ApplyEngine)

```

Tres fases separadas, cada una con su propio endpoint y guard de **RunPhase**:

Fase	Endpoint	Estado	Qué produce
Interpretación	POST /agent/interpret	interpreting → awaiting_confirmation	InterpretationDraft (contrato + acciones propuestas)
Confirmación	POST /agent/confirm	awaiting_confirmation → confirmed	CompiledPlan (skill_ir + frame + actions)
Ejecución	POST /agent/apply	confirmed → applying → completed/failed	FileOps + git commit + verify result

Terminal states: **completed, failed, cancelled.**

```
|
└── ApplyEngine (único mezclador) ─┘
```

Frontera de pureza (`graphir/boundary.py`): impide que claves del plano repositorio (`file_path`, `workspace`, `fs_*`) aparezcan dentro del grafo semántico.

Pipeline de Ejecución (ApplyEngine)



Componentes clave

Componente	Rol
StructuralIndex	Realidad estructural: <code>exists(cap)</code> , <code>resolve_path</code> , scan worktree
complete_structure()	Conciliador: dada intención + index → lifecycle por capability
GraphIRBuilder	Materializa StructuralIR + resolution → grafo semántico
ApplyEngine	Único punto de mezcla entre intención y realidad
ReactBackend	GraphIR → TSX code, con override de firmas reales
VerifyWorktree	<code>tsc --noEmit</code> post-apply, fail-closed

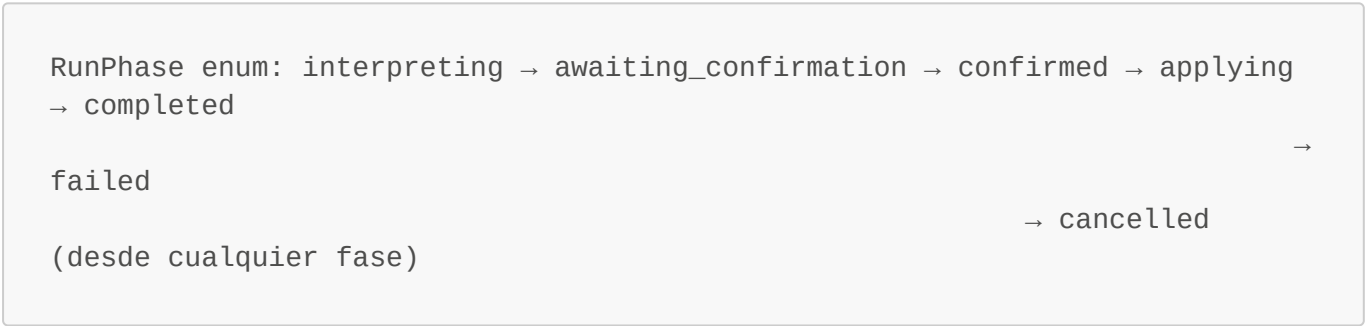
Sistema de Intención

Tres módulos que NO conocen la realidad del repo:

Módulo	Archivo	Input → Output
IntentInterpreter	<code>intent/interpreter.py</code>	Lenguaje natural + catálogo → <code>InterpretationDraft</code> (único LLM)
PlanCompiler	<code>intent/plan_compiler.py</code>	<code>ConfirmedIntent</code> + contrato → <code>CompiledPlan</code> (determinista, sin LLM)
Catalog	<code>catalog/loader.py</code>	<code>capability_catalog.json</code> → vocabulario + sinónimos

Anti-patrón: PlanCompiler NO llama `StructuralIndex`. `StructuralIndex` NO decide intención. `ApplyEngine` es el único mezclador.

Máquina de estados



Validada por `validate_transition()` en cada endpoint. 29 tests.

Orquestador (LangGraph)

`orchestrator/` — grafo con human-in-the-loop:



- Estado persistido en `AgentState` → `store.save_snapshot()`
- Reanudación via `POST /run/{id}/confirm` y `/apply`
- SSE events: `interpretation_ready`, `plan_preview_ready`, `result`

Phase 4.5 — Component Signature Extraction (✅ COMPLETED)

Problema: El renderer generaba tipos genéricos (`any[]`, `ReactNode`) causando `verify_failed` por type mismatch.

Solución: Extraer interfaces reales desde `.tsx` del worktree:

Capa	Archivo	Qué hace
Extractor	<code>signature/extractor.py</code>	Regex sobre <code>.tsx</code> : captura interface/type blocks + <code>extra_types</code> + <code>prop_names</code> + imports
Registry	<code>BackendConfig.component_signatures</code>	<code>{ComponentType: {props, extra_types, prop_names, imports}}</code>
Injection	<code>apply_engine.py</code>	Antes de <code>BackendConfig</code> , llama <code>extract_signatures(workspace)</code>
Override	<code>react_backend.py:_render_signature()</code>	Genera archivo: imports filtrados + <code>extra_types</code> + <code>props_block</code> + export
Prop filtering	<code>_render_children()</code> + constraint renderer	Filtra props contra <code>prop_names</code> conocidos

Resultado: `tsc --noEmit` pasa sin errores de tipo. Los componentes usan interfaces reales. Props no coincidentes se filtran (seguridad, no semántica — el binding semántico es Phase 5).

Phase 5 — Semantic Prop Binding (🚀 FUTURE)

Problema: Props filtradas = componente sin datos (`<Timeseries />` en vez de `<Timeseries data={...} />`).

Solución: `prop_mapper.py` traduce params de contrato (`metric: "revenue"`) a props de componente (`data: fetchTimeseries("revenue")`).

Objetivo	Detalle
<code>PARAM_ALIASES</code>	<code>metric</code> → <code>data</code> , <code>metrics</code> → <code>data</code> , <code>timeseries_metric</code> → <code>data</code>
<code>data_access.json</code>	Registro explícito: <code>{component: {prop: {imports, expression}}}</code>
Template resolution	<code>expression: "fetchTimeseries({metric})"</code> resuelto desde <code>contract_params</code>
Drift guards	4 reglas de validación contra inconsistencia contrato ↔ <code>data_access</code>
Import pruning	Post-pass opcional sobre imports de composición
BindingResult	Warnings estructurados por categoría (<code>semantic_drift</code> , <code>contract_mismatch</code> , etc.)

Plan detallado: `tmp/phase-5-prop-binding-plan.md`

Estructura del Proyecto

```

backend/
├── app/
│   ├── api/                # FastAPI endpoints (/agent/interpret,
│   │   │   /confirm, /apply)
│   │   ├── intent/        # IntentInterpreter, PlanCompiler, models,
│   │   │   state machine
│   │   │   ├── models.py  # InterpretationDraft, ConfirmedIntent,
│   │   │   │   CompiledPlan, RunPhase
│   │   │   ├── interpreter.py # IntentInterpreter (único LLM)
│   │   │   ├── plan_compiler.py # PlanCompiler (determinista)
│   │   │   └── llm_client.py # LLM client
│   │   ├── catalog/      # capability_catalog.json loader
│   │   └── state/        # RunState persistence (JSON files by
run_id)
│   ├── signature/        # Component signature extraction (Phase 4.5)
│   │   └── extractor.py  # Regex scanner para interfaces TSX
│   └── engine/           # ApplyEngine, StructuralIndex,
complete_structure
│   ├── apply_engine.py    # Pipeline principal
│   ├── structural_index.py # Worktree scanner + existence checks
│   └── structural_completion.py # Lifecycle conciliator
│   ├── graphir/          # Grafo semántico
│   │   ├── models.py     # GraphIR, FileOp, PipelineRoute
│   │   ├── boundary.py   # Pureza semántica
│   │   ├── pipeline.py   # GraphIRPipeline
│   │   ├── path_resolver.py # File path resolution
│   │   ├── backends/     # ReactBackend + generators
│   │   ├── constraint/   # Constraint Graph pipeline
│   │   └── structure/    # Resolver + canonicalizer
│   ├── config/           # Settings + feature flags
│   └── executor/         # Worktree management, file I/O
├── main.py               # FastAPI entry point
├── orchestrator/         # LangGraph orquestador
(interpret→confirm→apply)
├── ui/                   # Frontend web (single HTML + Vite)
├── tests/
│   ├── intent/           # 84 tests: interpreter, plan_compiler,
│   │   │   state_machine, E2E
│   │   └── e2e/          # 26 tests E2E de pipeline
│   ├── conftest.py
│   └── helpers.py
├── tmp/                  # Planes activos
│   └── phase-5-prop-binding-plan.md

```

Testing

Suite	Tests	Qué cubre
tests/intent/	84	Interpreter, PlanCompiler, state machine, E2E flow
tests/e2e/	26	Pipeline completo (remove/add KPI, update dashboard, etc.)
tests/ (core)	467	Unitarios + integración (2 skipped)

Ejecución:

```
REPO_ROOT=/path/to/repo pytest tests/ -q --ignore=tests/e2e
```

Configuración

Variable	Requerida	Default	Propósito
REPO_ROOT	Sí	—	Ruta al repositorio origen
RUNS_DIR	No	/opt/agent-repos/worktrees	Directorio de worktrees
ARTIFACTS_DIR	No	/opt/agent-repos/artifacts	Directorio de artefactos
LLM_BASE_URL	No	http://localhost:7000	URL del LLM
LLM_MODEL	No	Qwen/Qwen2.5-Coder-3B-Instruct	Modelo LLM

Feature flags en `config/feature_flags.py`: `structural_resolver`, `constraint_graph`, `constraint_graph_line_range`.

Feature Flags

```
FEATURE_FLAGS = {
    "structural_resolver": True,      # Resolución determinista de
    identidad
    "constraint_graph": True,        # Pipeline constraint-aware
}
```

Levantar el Sistema

1. vLLM (solo si no está corriendo)

```
cd /opt/agent-system
docker compose up -d vllm
```

Health check: `curl http://localhost:7000/health`

2. Todos los servicios

```
cd /opt/agent-system
docker compose up -d
```

Health checks:

```
curl -s http://localhost:8000/health # Backend
curl -s http://localhost:9000/health # Orchestrator
curl -s http://localhost:7000/health # vLLM
curl -s -o /dev/null -w "%{http_code}" http://localhost:5173/ # UI
```

Logs:

```
docker compose logs -f # Todos
docker logs agent-orchestrator -f # Solo orchestrator
docker logs agent-backend -f # Solo backend
```

3. Refrescar sistema (tras cambios en código)

```
cd /opt/agent-system
docker compose up -d --build
```

4. Detener sistema

```
cd /opt/agent-system
docker compose down
```

Los datos persisten (worktrees, artifacts, snapshots).

API Surface

Puerto	Servicio
8000	Backend API
9000	Orchestrator API
7000	vLLM inference

Puerto	Servicio
5173	UI frontend

Via UI

1. Abrir `http://localhost:5173`
2. Escribir tarea en lenguaje natural
3. Presionar "Run"
4. Ver SSE streaming en vivo

Via CLI

Nuevo flujo (interpret → confirm → apply):

```
# 1. Interpretar intención
INTERP=$(curl -s -X POST http://localhost:8000/agent/interpret \
  -H "Content-Type: application/json" \
  -d '{"run_id":"demo","message":"remove KPI row from dashboard"}')
echo "$INTERP" | python3 -m json.tool

# 2. Confirmar plan
CONFIRM=$(curl -s -X POST http://localhost:8000/agent/confirm \
  -H "Content-Type: application/json" \
  -d '{"run_id":"demo"}')
echo "$CONFIRM" | python3 -m json.tool

# 3. Aplicar
APPLY=$(curl -s -X POST http://localhost:8000/agent/apply \
  -H "Content-Type: application/json" \
  -d '{"run_id":"demo","confirmed_intent":{"...}}')
echo "$APPLY" | python3 -m json.tool

# 4. Consultar run
curl -s http://localhost:8000/runs/demo | python3 -m json.tool
```

Via Orchestrator (LangGraph):

```
# Crear run
RUN_OUT=$(curl -s -X POST http://localhost:9000/run \
  -H "Content-Type: application/json" \
  -d '{"task":"create file hello.txt with content HELLO"}')
RUN_ID=$(echo "$RUN_OUT" | python3 -c "import sys,json;print(json.load(sys.stdin)['run_id'])")

# Ver streaming
curl -N http://localhost:9000/stream/$RUN_ID

# Ver snapshot persistido
curl -s http://localhost:9000/runs/$RUN_ID | python3 -m json.tool
```

```
# Listar todos los runs
curl -s http://localhost:9000/runs | python3 -m json.tool
```

MCP Server

El sistema incluye un servidor MCP (Model Context Protocol) en `backend/mcp-server/server.py`.

Herramientas

Tool	Params	Descripción
agent_plan	prompt: str	Generar plan de ejecución
agent_apply	run_id: str, plan: dict, dry_run: bool	Ejecutar plan en sandbox
get_run	run_id: str	Obtener metadata del run
agent_review	run_id: str	Inspeccionar plan, diff, ejecución
agent_run	prompt: str, dry_run: bool	One-shot plan + apply
agent_approve	run_id: str	Aprobar y mergear cambios
agent_reject	run_id: str	Rechazar y limpiar

Conexión

```
{
  "mcpServers": {
    "agent-runtime": {
      "command": "python",
      "args": ["/opt/agent-system/backend/mcp-server/server.py"],
      "env": { "PYTHONPATH": "/opt/agent-system/backend" }
    }
  }
}
```

Backup Docker compose

```
sudo bash /opt/agent-system/scripts/backup_system.sh
```

Configuración del Repositorio

El backend clona/usa un repo Git en **REPO_ROOT**. Cada run:

1. Crea un branch **agent-`{run_id[:8]}`** desde **master**
2. Crea un worktree en **`{RUNS_DIR}/{run_id}/workspace/`**
3. Aplica operaciones (create, modify, delete)
4. Hace **`git add -A`** y **`git commit -m "agent:{run_id}"`**
5. Genera diff

Para limpiar worktrees antiguos:

```
curl -X POST http://localhost:8000/maintenance/cleanup
```

Limitaciones del Modelo 3B

El sistema usa **Qwen/Qwen2.5-Coder-3B-Instruct**, un modelo pequeño para ejecución local en GPU de consumo.

- **Razonamiento superficial:** planes simples, pocos pasos
- **Contexto limitado:** 4096 tokens máximo
- **Refactors grandes no confiables:** tareas multi-archivo tienden a ser inconsistentes
- **Alucinación de paths:** puede inventar rutas que no existen
- **Sin conocimiento del repositorio:** no entiende la estructura actual sin contexto explícito

La estrategia del sistema para mitigar estas limitaciones no es pedirle más al LLM, sino rodearlo con capas deterministas: registry validation, GraphIR pipeline, ReactBackend, executor dumb. El LLM nunca decide directamente qué archivos crear ni qué contenido escribir — solo selecciona capability y rellena parámetros.